

BELLABEAT FITNESS TRACKER

Python Analysis — Complete Google Colab Notebook

Fixed Version • 19 cells • 13 charts • 9 Power BI exports

Errors fixed in this version

- ✗FileNotFoundError: user_segments.csv → ✔Derived inside notebook from user_summary
 - ✗FileNotFoundError: hourly_combined.csv → ✔Built from peak_hours.csv already uploaded
 - ✗MessageError: drive.mount credential failed → ✔Removed — upload files directly
- ✔All 9 pbi_ CSV files exported automatically in Cell 18

SECTION 1 — Setup & Configuration

CELL 01

Install & Import All Libraries

⚠ Note: Run this cell first every Colab session. All libraries are pre-installed in Colab so pip lines are commented out.

```
# — Installs (pre-installed in Colab — uncomment ONLY if you get
ModuleNotFoundError) —
# !pip install pandas numpy matplotlib seaborn scipy --quiet

# — Core libraries


---


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
from scipy.stats import pearsonr, ttest_ind, gaussian_kde
import os, zipfile, warnings
warnings.filterwarnings("ignore")

# — Global colour palette


---


BLUE = '#2563EB'
NAVY = '#1F3864'
RED = '#EF4444'
ORANGE = '#F97316'
GREEN = '#16A34A'
PURPLE = '#4F46E5'
PALETTE = ['#1F3864', '#2563EB', '#60A5FA', '#BFDDBE']

# — Chart style


---


sns.set_theme(style='whitegrid', font_scale=1.05)
plt.rcParams['figure.dpi'] = 150
plt.rcParams['font.family'] = 'DejaVu Sans'
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.titlesize'] = 13

# Day order constant used throughout notebook
DAY_ORDER =
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

print('✔ All libraries loaded successfully')
```

CELL 02

Load All 7 CSV Files (No drive.mount needed)

Files uploaded via the Files panel are at /content/filename.csv automatically. No drive.mount needed — that code is gone.

```
# — Load all 7 CSV files


---


```

```

# These files must be uploaded first via the Files panel (left sidebar → upload
icon).
# After upload they sit at /content/filename.csv – that is the path below.

activity = pd.read_csv('master_user_activity.csv')
user_sum = pd.read_csv('user_summary.csv')
daytrend = pd.read_csv('day_of_week_trends.csv')
hourly    = pd.read_csv('peak_hours.csv')          # also becomes
pbi_hourly_combined
scatter   = pd.read_csv('steps_vs_calories.csv')
sleep     = pd.read_csv('sleep_analysis.csv')
weekly    = pd.read_csv('weekly_trend.csv')

# — user_segments is NOT a file – built in Cell 09 from user_sum
# — hourly_combined is NOT a file – hourly is used directly

# — Parse dates

activity['activity_date'] = pd.to_datetime(activity['activity_date'])
weekly['week_start']      = pd.to_datetime(weekly['week_start'])

# — Enforce day-of-week category order

daytrend['day_of_week'] = pd.Categorical(
    daytrend['day_of_week'], categories=DAY_ORDER, ordered=True)
daytrend = daytrend.sort_values('day_of_week').reset_index(drop=True)

# — Confirm all tables loaded

tables = {'master_user_activity':activity, 'user_summary':user_sum,
          'day_of_week_trends':daytrend,   'peak_hours':hourly,
          'steps_vs_calories':scatter,     'sleep_analysis':sleep,
          'weekly_trend':weekly}
print(f"{'Table':<28} {'Rows':>7} {'Cols':>5}")
print('-'*45)
for name, df in tables.items():
    print(f'{name:<28} {len(df):>7,} {df.shape[1]:>5}')
print(f"\nUnique users : {activity['user_id'].nunique()}")
print(f"Date range   : {activity['activity_date'].min().date()} →
{activity['activity_date'].max().date()}")
print('\n✔ All 7 files loaded – ready to analyse')

```

Table	Rows	Cols
master_user_activity	940	20
user_summary	33	12
day_of_week_trends	7	6
peak_hours	24	4
steps_vs_calories	863	7
sleep_analysis	24	7
weekly_trend	5	5

Unique users : 33

Date range : 2016-04-12 → 2016-05-12

✔ All 7 files loaded — ready to analyse

SECTION 2 — Exploratory Data Analysis (EDA)

CELL 03

Descriptive Statistics — Master Table

⚠ Note: Always run descriptive stats first — reveals range, mean, std dev, and outliers before any plotting.

```
# — Key numeric columns
NUM_COLS = [
    'total_steps', 'total_distance', 'calories',
    'very_active_minutes', 'fairly_active_minutes',
    'lightly_active_minutes', 'sedentary_minutes',
    'total_active_minutes'
]
print("=== DAILY ACTIVITY — DESCRIPTIVE STATISTICS ===")
print(activity[NUM_COLS].describe().round(2).to_string())

# — Sleep stats (only rows that have sleep data)
SLEEP_COLS = ['total_minutes_asleep', 'total_time_in_bed',
              'minutes_away_in_bed', 'sleep_efficiency_pct']
sleep_rows = activity.dropna(subset=['total_minutes_asleep'])
print("\n=== SLEEP — DESCRIPTIVE STATISTICS ===")
print(sleep_rows[SLEEP_COLS].describe().round(2).to_string())

# — Null counts
nulls = activity[NUM_COLS + SLEEP_COLS].isnull().sum()
print("\n=== NULL COUNTS (only columns with nulls shown) ===")
print(nulls[nulls > 0])
print(f"\nRows with sleep data : {sleep_rows.shape[0]:,} / {len(activity):,}")
print(f"Users with sleep data: {sleep_rows['user_id'].nunique()} / {activity['user_id'].nunique()}")
```

```
=== DAILY ACTIVITY — DESCRIPTIVE STATISTICS ===
      total_steps  total_distance  calories  very_active_minutes
fairly_active_minutes  lightly_active_minutes  sedentary_minutes
total_active_minutes
count      940.00      940.00      940.00      940.00
940.00      940.00      940.00      940.00      940.00
mean      7637.91      5.49      2303.61      21.16
13.56      192.81      991.21      227.54
std      5087.15      3.92      718.17      32.84
19.99      109.17      301.27      121.78
min      0.00      0.00      0.00      0.00
0.00      0.00      0.00      0.00
25%      3789.75      2.62      1828.50      0.00
0.00      127.00      729.75      146.75
50%      7405.50      5.24      2134.00      4.00
6.00      199.00      1057.50      247.00
75%      10727.00      7.71      2793.25      32.00
19.00      264.00      1229.50      317.25
max      36019.00      28.03      4900.00      210.00
143.00      518.00      1440.00      552.00
```

```
=== SLEEP — DESCRIPTIVE STATISTICS ===
      total_minutes_asleep  total_time_in_bed  minutes_away_in_bed
sleep_efficiency_pct
```

count	410.00	410.00	410.00
410.00			
mean	419.17	458.48	39.31
91.65			
std	118.64	127.46	46.71
8.73			
min	58.00	61.00	0.00
49.80			
25%	361.00	403.75	17.00
91.20			
50%	432.50	463.00	25.50
94.30			
75%	490.00	526.00	40.00
96.10			
max	796.00	961.00	371.00
100.00			

=== NULL COUNTS (only columns with nulls shown) ===

```
total_minutes_asleep    530
total_time_in_bed       530
minutes_awake_in_bed    530
sleep_efficiency_pct     530
dtype: int64
```

Rows with sleep data : 410 / 940

Users with sleep data: 24 / 33

CELL 04

Data Participation — Chart 1 of 13

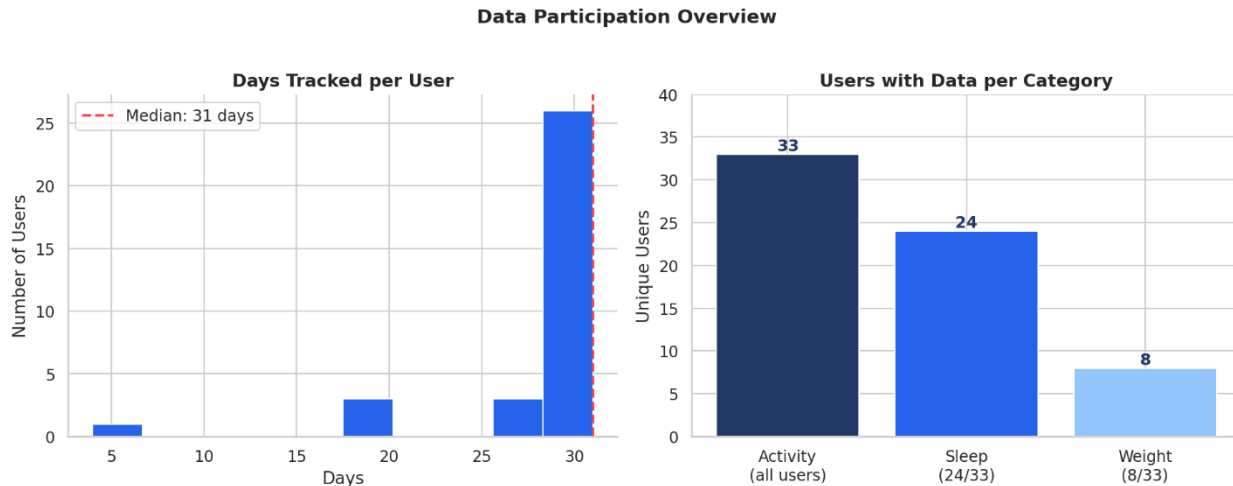
```
days_tracked =
activity.groupby('user_id')['activity_date'].nunique().sort_values()

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Left: days tracked histogram
axes[0].hist(days_tracked.values, bins=10, color=BLUE, edgecolor='white',
linewidth=0.9)
axes[0].axvline(days_tracked.median(), color=RED, linestyle='--', linewidth=1.8,
label=f'Median: {days_tracked.median():.0f} days')
axes[0].set_title('Days Tracked per User')
axes[0].set_xlabel('Days'); axes[0].set_ylabel('Number of Users')
axes[0].legend()

# Right: data completeness bar
labels = ['Activity\n(all users)', 'Sleep\n(24/33)', 'Weight\n(8/33)']
counts = [
    activity['user_id'].nunique(),
    activity.dropna(subset=['total_minutes_asleep'])['user_id'].nunique(),
    activity.dropna(subset=['bmi'])['user_id'].nunique()
]
bars = axes[1].bar(labels, counts, color=[NAVY, BLUE, '#93C5FD'],
edgecolor='white')
for bar in bars:
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+.4,
str(int(bar.get_height())), ha='center', fontsize=12,
fontweight='bold', color=NAVY)
axes[1].set_title('Users with Data per Category')
axes[1].set_ylabel('Unique Users'); axes[1].set_ylim(0, 40)
```

```
plt.suptitle('Data Participation Overview', fontsize=14, fontweight='bold',
y=1.02)
plt.tight_layout()
plt.savefig('chart01_participation.png', bbox_inches='tight', dpi=150)
plt.show()
```



□ **Insight:** Only 24/33 users logged sleep and 8/33 logged weight — state this as a data limitation in your presentation. It shows analytical maturity.

CELL 05

Distribution Plots — 6 Key Metrics — Chart 2 of 13

⚠ **Note:** Histograms with KDE reveal skewness, outliers, and bimodal patterns. Required for any EDA section.

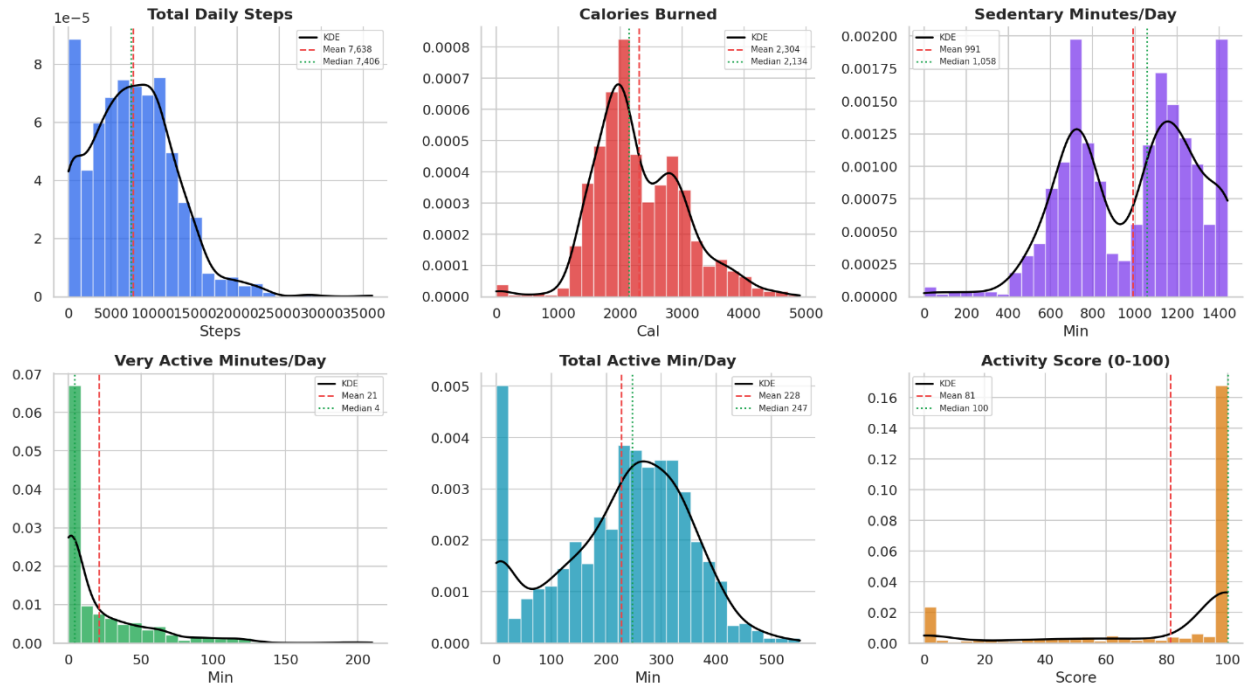
```
metrics = {
    'total_steps': ('Total Daily Steps', 'Steps', '#2563EB'),
    'calories': ('Calories Burned', 'Cal', '#DC2626'),
    'sedentary_minutes': ('Sedentary Minutes/Day', 'Min', '#7C3AED'),
    'very_active_minutes': ('Very Active Minutes/Day', 'Min', '#16A34A'),
    'total_active_minutes': ('Total Active Min/Day', 'Min', '#0891B2'),
    'activity_score': ('Activity Score (0-100)', 'Score', '#D97706'),
}

fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes = axes.flatten()

for i, (col, (title, xlabel, color)) in enumerate(metrics.items()):
    data = activity[col].dropna()
    axes[i].hist(data, bins=25, color=color, alpha=0.75,
                 edgecolor='white', linewidth=0.7, density=True)
    xs = np.linspace(data.min(), data.max(), 200)
    axes[i].plot(xs, gaussian_kde(data)(xs), color="black", linewidth=1.8,
label="KDE")
    axes[i].axvline(data.mean(), color=RED, linestyle="--", linewidth=1.4,
                    label=f"Mean {data.mean():.0f}")
    axes[i].axvline(data.median(), color=GREEN, linestyle=":", linewidth=1.4,
                    label=f"Median {data.median():.0f}")
    axes[i].set_title(title)
    axes[i].set_xlabel(xlabel)
    axes[i].legend(fontsize=7)
```

```
plt.suptitle('Distribution of Key Daily Metrics', fontsize=15, fontweight='bold',
y=1.01)
plt.tight_layout()
plt.savefig('chart02_distributions.png', bbox_inches='tight', dpi=150)
plt.show()
```

Distribution of Key Daily Metrics



SECTION 3 — Statistical Analysis

CELL 06

Correlation Matrix — Chart 3 of 13

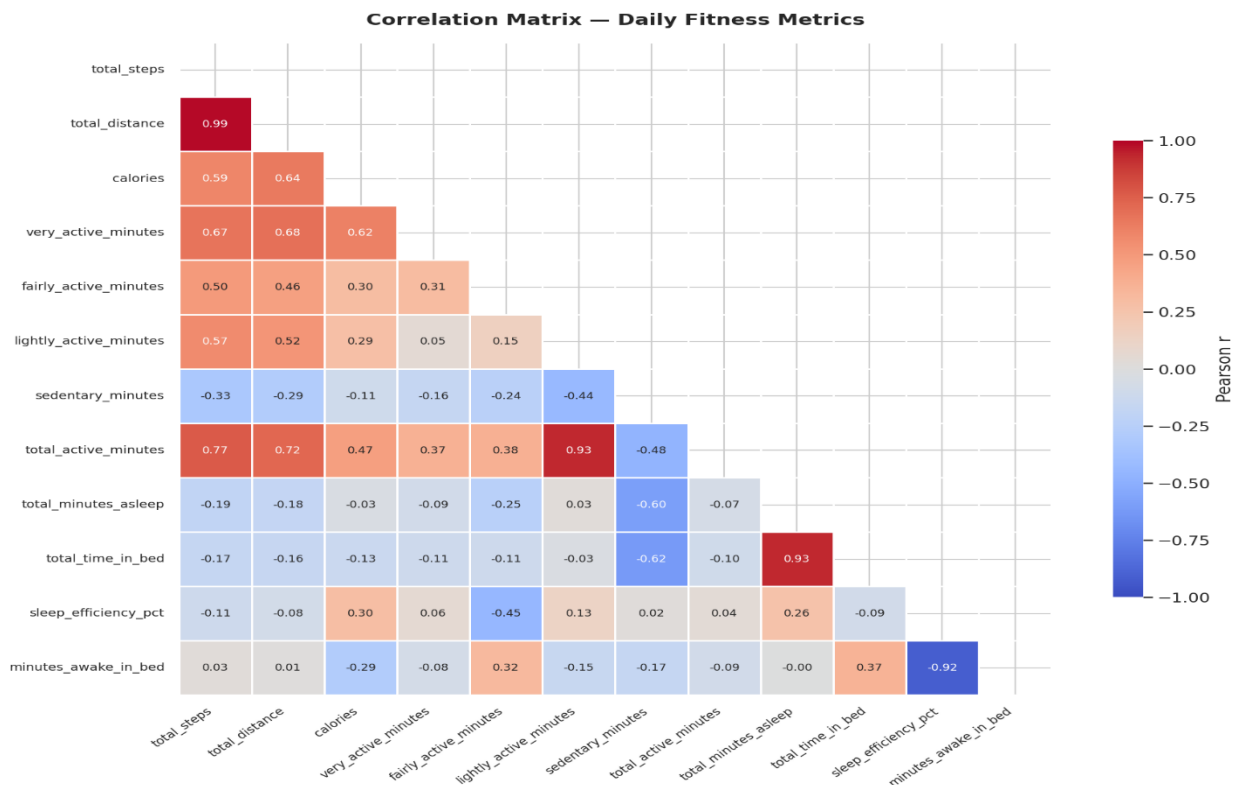
□ **Insight:** *The correlation matrix is the centrepiece of a data analyst portfolio — shows all relationships at once and demonstrates statistical thinking.*

```
corr_cols = [
    'total_steps', 'total_distance', 'calories', 'very_active_minutes',
    'fairly_active_minutes', 'lightly_active_minutes', 'sedentary_minutes',
    'total_active_minutes', 'total_minutes_asleep',
    'total_time_in_bed', 'sleep_efficiency_pct', 'minutes_awake_in_bed'
]
corr_data = activity[corr_cols].dropna(thresh=8)
corr_matrix = corr_data.corr(method="pearson")

fig, ax = plt.subplots(figsize=(13, 10))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

sns.heatmap(
    corr_matrix, mask=mask, ax=ax,
    cmap='coolwarm', vmin=-1, vmax=1, center=0,
    annot=True, fmt=".2f", annot_kws={"size":8},
    linewidths=0.5, square=True,
    cbar_kws={'shrink':0.7, 'label':'Pearson r'}
)
ax.set_title('Correlation Matrix - Daily Fitness Metrics', fontsize=14,
            fontweight='bold', pad=16)
ax.set_xticklabels(ax.get_xticklabels(), rotation=40, ha='right', fontsize=9)
ax.set_yticklabels(ax.get_yticklabels(), rotation=0, fontsize=9)
plt.tight_layout()
plt.savefig('chart03_correlation_matrix.png', bbox_inches='tight', dpi=150)
plt.show()

# Print top correlations
pairs = (corr_matrix
        .where(np.tril(np.ones(corr_matrix.shape), k=-1).astype(bool))
        .stack().reset_index())
pairs.columns = ["var1", "var2", "r"]
pairs = pairs.reindex(pairs['r'].abs().sort_values(ascending=False).index)
print('Top 8 strongest correlations:')
print(pairs.head(8)[['var1', 'var2', 'r']].to_string(index=False))
```

CELL 07

Steps vs Calories Scatter + Regression — Chart 4 of 13

```
valid = scatter[(scatter['total_steps']>0) & (scatter['calories']>0)].copy()
r_val, p_val = pearsonr(valid['total_steps'], valid['calories'])
slope, intercept, *_ = stats.linregress(valid['total_steps'], valid['calories'])

seg_colors = {'1_Sedentary': '#BFDDBFE', '2_Lightly Active': '#3B82F6',
              '3_Fairly Active': '#1D4ED8', '4_Very Active': '#1F3864'}

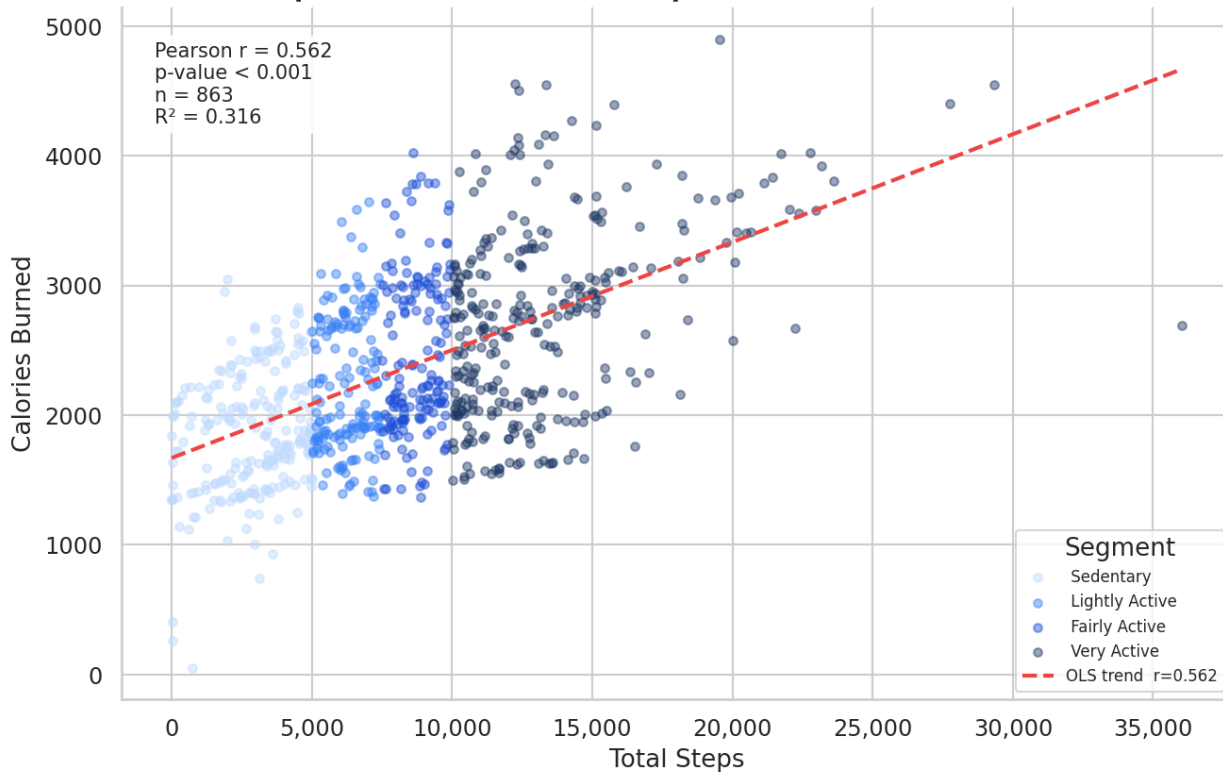
fig, ax = plt.subplots(figsize=(9, 6))
for seg, grp in valid.groupby('daily_activity_segment'):
    ax.scatter(grp['total_steps'], grp['calories'],
               color=seg_colors.get(seg, 'grey'), alpha=0.45, s=18,
               label=seg.replace('_', ' ').rstrip('1234_'))

x_range = np.linspace(valid["total_steps"].min(), valid["total_steps"].max(), 300)
ax.plot(x_range, slope*x_range+intercept, color=RED, linewidth=2,
        linestyle='--', label=f'OLS trend  r={r_val:.3f}')

ax.text(0.03, 0.95,
        f'Pearson r = {r_val:.3f}\nnp-value < 0.001\nn = {len(valid):,}\nnR² = {r_val**2:.3f}',
        transform=ax.transAxes, fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.85))

ax.set_title(f'Steps vs Calories - R²={r_val**2:.2f}, positive correlation confirmed')
ax.set_xlabel('Total Steps'); ax.set_ylabel('Calories Burned')
ax.legend(title='Segment', fontsize=8)
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:,.0f}'))
plt.tight_layout()
plt.savefig('chart04_steps_vs_calories.png', bbox_inches='tight', dpi=150)
plt.show()
print(f'r={r_val:.4f}  R²={r_val**2:.4f}  p={p_val:.2e}')
```

Steps vs Calories — $R^2=0.32$, positive correlation confirmed



CELL 08

Welch T-Test — Active vs Sedentary Calories — Chart 5 of 13

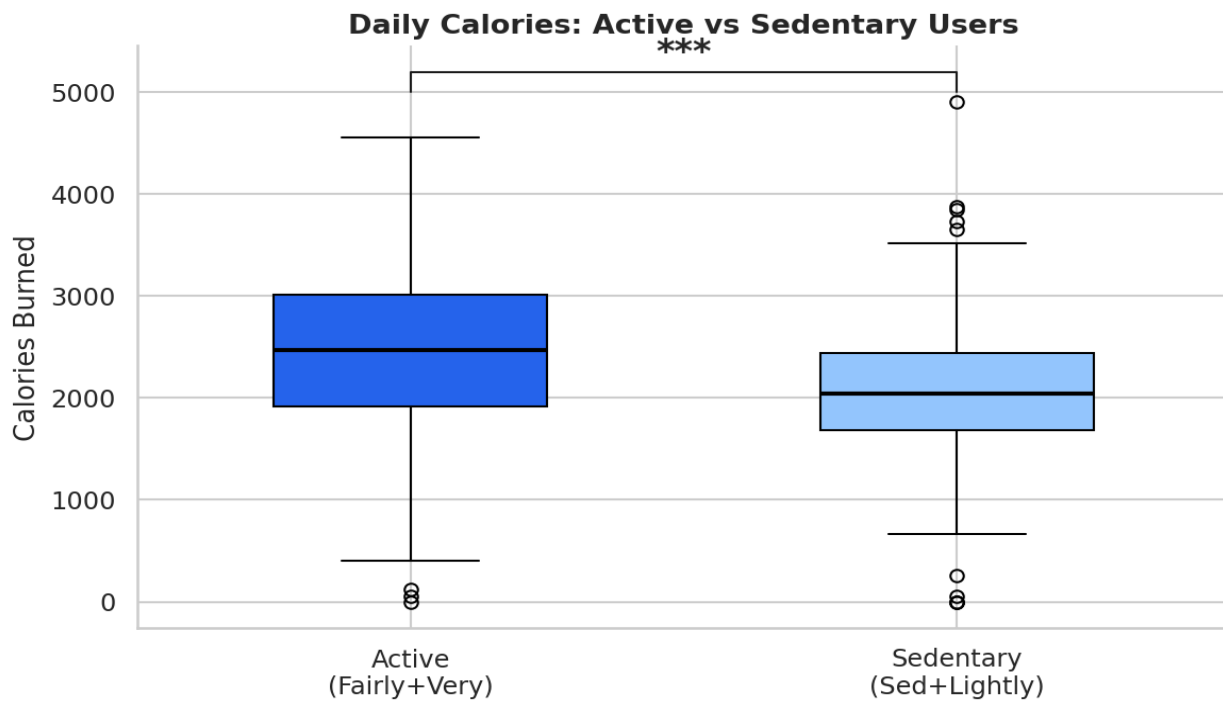
Note: Hypothesis testing shows hiring managers you understand inferential statistics, not just descriptive.

```
active_ids = user_sum[user_sum['user_segment'].isin(
    ['Fairly Active', 'Very Active'])]['user_id']
sed_ids = user_sum[user_sum['user_segment'].isin(
    ['Sedentary', 'Lightly Active'])]['user_id']
active_cal = activity[activity['user_id'].isin(active_ids)]['calories'].dropna()
sed_cal = activity[activity['user_id'].isin(sed_ids)]['calories'].dropna()
t_stat, p_val = ttest_ind(active_cal, sed_cal, equal_var=False)

fig, ax = plt.subplots(figsize=(8, 5))
bp = ax.boxplot([active_cal, sed_cal],
    labels=['Active\n(Fairly+Very)', 'Sedentary\n(Sed+Lightly)'],
    patch_artist=True, widths=0.5,
    medianprops=dict(color='black', linewidth=2))
bp['boxes'][0].set_facecolor(BLUE)
bp['boxes'][1].set_facecolor('#93C5FD')

y_max = max(active_cal.max(), sed_cal.max())
ax.plot([1, 1.2, 1.6, 1.6, 1.2, 1], [y_max*1.02, y_max*1.06, y_max*1.06, y_max*1.02], 'k-', linewidth=1)
sig = '***' if p_val < 0.001 else '**' if p_val < 0.01 else '*' if p_val < 0.05 else 'ns'
ax.text(1.5, y_max*1.08, sig, ha='center', fontsize=16, fontweight='bold')
ax.set_title('Daily Calories: Active vs Sedentary Users')
ax.set_ylabel('Calories Burned')
plt.tight_layout()
plt.savefig('chart05_ttest_calories.png', bbox_inches='tight', dpi=150)
plt.show()
print(f't={t_stat:.3f} p={p_val:.2e} {"SIGNIFICANT" if p_val < 0.05 else "ns"}')
print(f'Active mean : {active_cal.mean():.0f} cal/day')
```

```
print(f'Sedentary mean: {sed_cal.mean():,.0f} cal/day')
print(f'Difference      : {active_cal.mean()-sed_cal.mean():,.0f} cal/day')
```



SECTION 4 — Activity Analysis

CELL 09

User Segmentation — Donut + Bar — Chart 6 of 13

Segments are built here from user_summary.csv — fixing the FileNotFoundError for user_segments.csv.

```
# — Build segment counts from user_summary (no separate file needed)

SEG_ORDER = ['Sedentary', 'Lightly Active', 'Fairly Active', 'Very Active']
SEG_COLORS = ['#BFDDBFE', '#60A5FA', '#2563EB', '#1F3864']

seg_counts = user_sum['user_segment'].value_counts().reindex(SEG_ORDER).dropna()

fig, axes = plt.subplots(1, 2, figsize=(13, 6))

# Left: Donut


---


wedges, texts, autotexts = axes[0].pie(
    seg_counts.values, labels=seg_counts.index,
    autopct='%1.0f%%', colors=SEG_COLORS,
    startangle=140, wedgeprops=dict(width=0.55), pctdistance=0.78)
for t in autotexts: t.set_fontsize(11); t.set_fontweight('bold')
for t in texts: t.set_fontsize(10)
axes[0].set_title('User Activity Segments\n(by avg daily steps)', pad=14)

# Right: Grouped bar


---

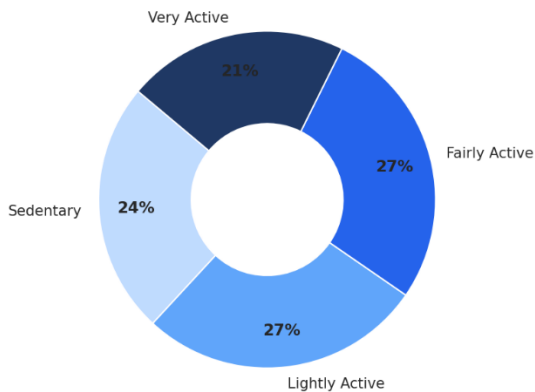

seg_stats = (user_sum.groupby('user_segment')
             [['avg_daily_steps', 'avg_daily_calories']]
             .mean().reindex(SEG_ORDER).dropna())
x = np.arange(len(seg_stats)); w = 0.35
axes[1].bar(x-w/2, seg_stats['avg_daily_steps']/1000,
            width=w, color=BLUE, label='Avg Steps (000s)')
axes[1].bar(x+w/2, seg_stats['avg_daily_calories']/100,
            width=w, color=ORANGE, label='Avg Calories (00s)')
axes[1].set_xticks(x)
axes[1].set_xticklabels(seg_stats.index, rotation=20, ha='right')
axes[1].set_title('Steps & Calories by Segment')
axes[1].legend()

plt.suptitle('User Activity Segmentation', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('chart06_user_segments.png', bbox_inches='tight', dpi=150)
plt.show()

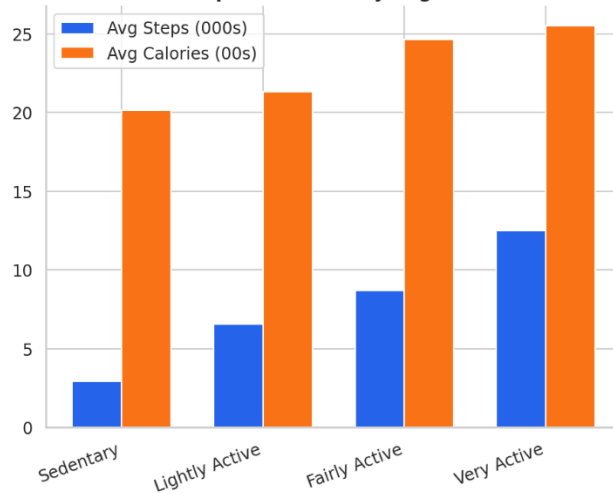
print(user_sum.groupby('user_segment')[
    ['avg_daily_steps', 'avg_daily_calories', 'avg_sedentary_min', 'avg_activity_score']
    ].mean().round(0).reindex(SEG_ORDER).to_string())
```

User Activity Segmentation

**User Activity Segments
(by avg daily steps)**



Steps & Calories by Segment



CELL 10

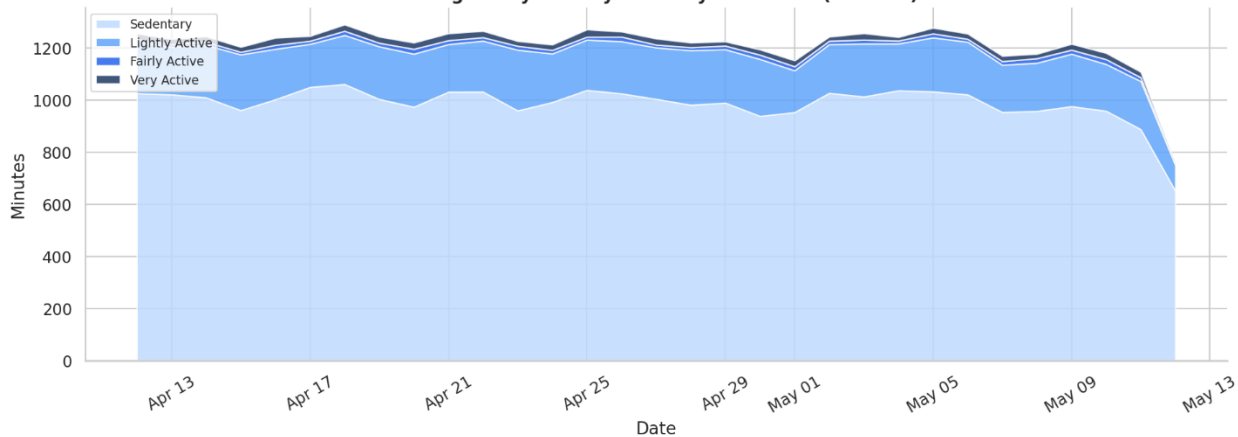
Active Minutes Stacked Area Chart — Chart 7 of 13

```
daily_int = activity.groupby('activity_date')[[
    'very_active_minutes', 'fairly_active_minutes',
    'lightly_active_minutes', 'sedentary_minutes']].mean().reset_index()

fig, ax = plt.subplots(figsize=(13, 5))
cols_a = ['sedentary_minutes', 'lightly_active_minutes',
          'fairly_active_minutes', 'very_active_minutes']
colors_a = ['#BFDDBFE', '#60A5FA', '#2563EB', '#1F3864']
labels_a = ['Sedentary', 'Lightly Active', 'Fairly Active', 'Very Active']

ax.stackplot(daily_int["activity_date"],
             [daily_int[c] for c in cols_a],
             labels=labels_a, colors=colors_a, alpha=0.85)
ax.set_title('Average Daily Activity Intensity Over Time (Stacked)')
ax.set_xlabel('Date'); ax.set_ylabel('Minutes')
ax.legend(loc='upper left', fontsize=9)
ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%b %d'))
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig('chart07_stacked_activity.png', bbox_inches='tight', dpi=150)
plt.show()
```

Average Daily Activity Intensity Over Time (Stacked)



SECTION 5 — Temporal & Hourly Analysis

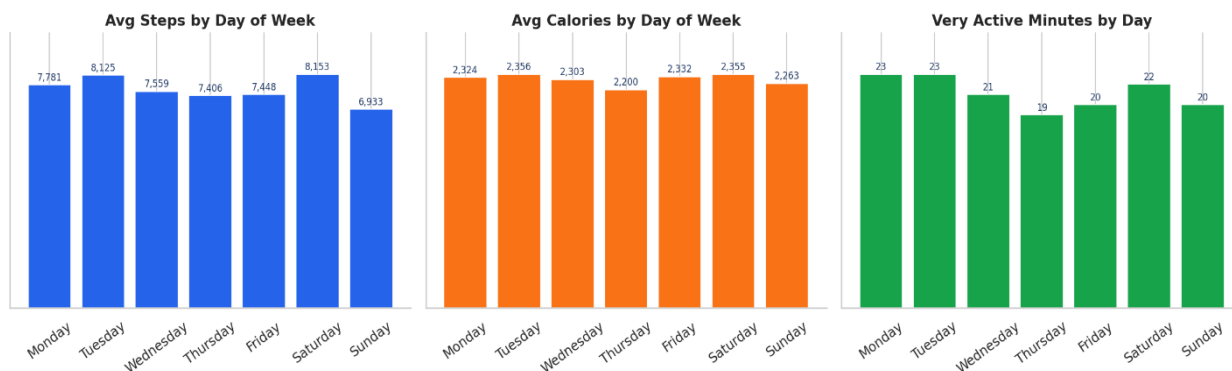
CELL 11

Day-of-Week Trends — 3 Charts — Chart 8 of 13

```
def bar_labeled(ax, x, y, color, title, ylabel):
    bars = ax.bar(x, y, color=color, edgecolor="white", linewidth=0.8)
    for bar in bars:
        ax.text(bar.get_x()+bar.get_width()/2,
                bar.get_height()+y.max()*0.01,
                f'{bar.get_height():,.0f}',
                ha='center', va='bottom', fontsize=8, color=NAVY)
    ax.set_title(title); ax.set_ylabel(ylabel)
    ax.set_ylim(0, y.max()*1.18)
    ax.tick_params(axis='x', rotation=35)
    ax.yaxis.set_visible(False) # data labels make Y axis redundant

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
bar_labeled(axes[0], daytrend['day_of_week'], daytrend['avg_steps'],
            BLUE, 'Avg Steps by Day of Week', 'Steps')
bar_labeled(axes[1], daytrend['day_of_week'], daytrend['avg_calories'],
            ORANGE, 'Avg Calories by Day of Week', 'Calories')
bar_labeled(axes[2], daytrend['day_of_week'], daytrend['avg_very_active_min'],
            GREEN, 'Very Active Minutes by Day', 'Minutes')
plt.suptitle('Activity Patterns by Day of Week', fontsize=14, fontweight='bold',
y=1.02)
plt.tight_layout()
plt.savefig('chart08_day_of_week.png', bbox_inches='tight', dpi=150)
plt.show()
print(f"Most active day :
{daytrend.loc[daytrend['avg_steps'].idxmax(), 'day_of_week']}")
print(f"Least active day :
{daytrend.loc[daytrend['avg_steps'].idxmin(), 'day_of_week']}")
print(f"Step gap : {daytrend['avg_steps'].max() -
daytrend['avg_steps'].min():,.0f}")
```

Activity Patterns by Day of Week



CELL 12

Peak Activity Hours — Area Chart — Chart 9 of 13

```
fig, ax = plt.subplots(figsize=(13, 5))
ax.fill_between(hourly['hour_of_day'], hourly['avg_steps'], alpha=0.12,
color=BLUE)
ax.plot(hourly['hour_of_day'], hourly['avg_steps'],
        color=NAVY, linewidth=2.5, marker='o', markersize=5)

# Shade engagement windows

ax.axvspan(6, 9, alpha=0.08, color='green', label='Morning window 6-9am')
```

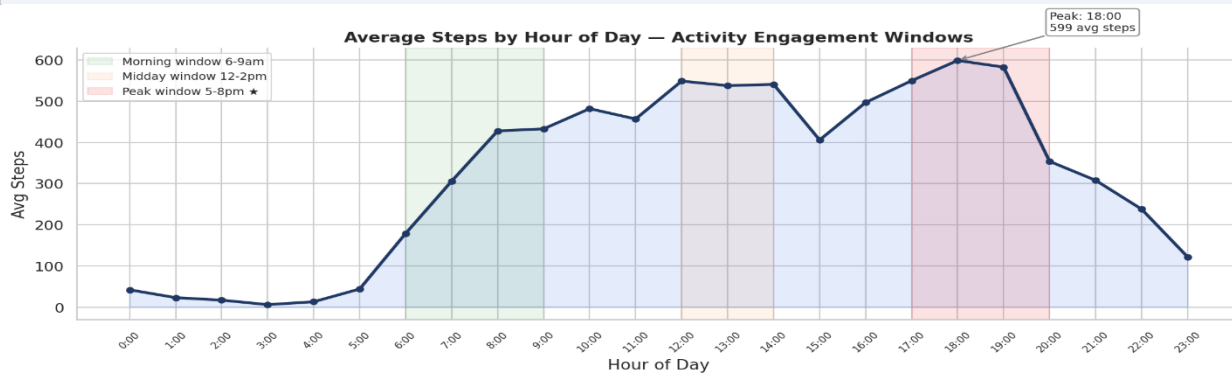
```

ax.axvspan(12, 14, alpha=0.08, color=ORANGE, label='Midday window 12-2pm')
ax.axvspan(17, 20, alpha=0.15, color=RED, label='Peak window 5-8pm ★')

peak_hr = hourly.loc[hourly['avg_steps'].idxmax(),'hour_of_day']
peak_val = hourly.loc[hourly['avg_steps'].idxmax(),'avg_steps']
ax.annotate(f'Peak: {peak_hr}:00\n{peak_val:,.0f} avg steps',
            xy=(peak_hr, peak_val), xytext=(peak_hr+2, peak_val*1.12),
            arrowprops=dict(arrowstyle='->', color='grey', lw=1.2),
            fontsize=9,
            bbox=dict(boxstyle='round,pad=0.3',facecolor='white',
                      edgcolor='grey',alpha=0.9))

ax.set_xticks(range(24))
ax.set_xticklabels([f'{h}:00' for h in range(24)], rotation=45, fontsize=8)
ax.set_title('Average Steps by Hour of Day — Activity Engagement Windows')
ax.set_xlabel('Hour of Day'); ax.set_ylabel('Avg Steps')
ax.legend(fontsize=9)
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:,.0f}'))
plt.tight_layout()
plt.savefig('chart09_peak_hours.png', bbox_inches='tight', dpi=150)
plt.show()
morning = hourly[hourly['hour_of_day'].between(6,9)]['avg_steps'].mean()
evening = hourly[hourly['hour_of_day'].between(17,20)]['avg_steps'].mean()
print(f'Morning avg (6-9am) : {morning:,.0f} steps')
print(f'Evening avg (5-8pm) : {evening:,.0f} steps ({evening/morning:.1f}x more active)')

```



CELL 13

Weekly Trend — Dual Axis Line Chart — Chart 10 of 13

```

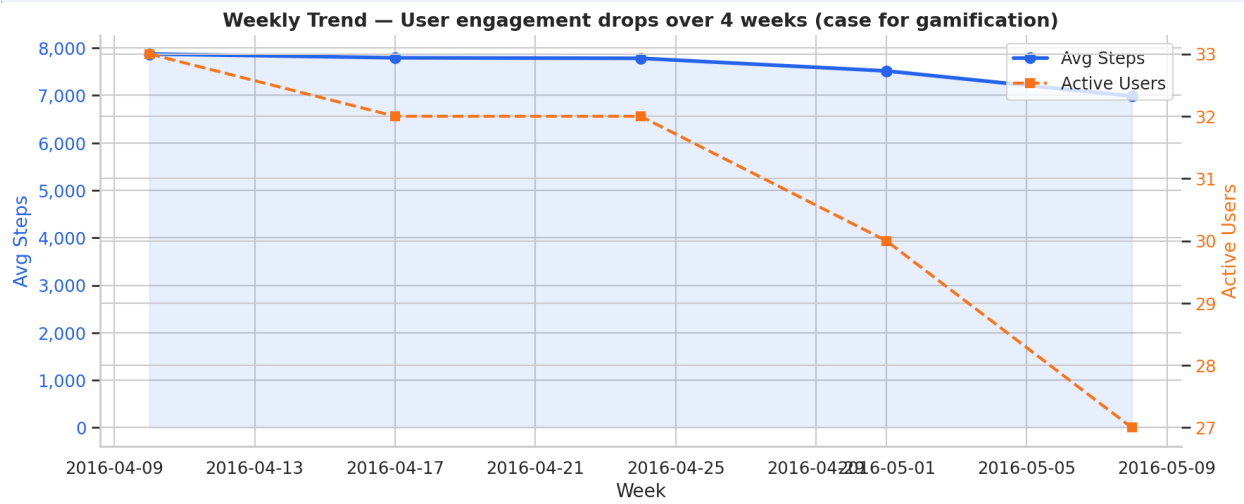
fig, ax1 = plt.subplots(figsize=(12, 5))
ax1.fill_between(weekly['week_start'], weekly['avg_steps'], alpha=0.1, color=BLUE)
ax1.plot(weekly['week_start'], weekly['avg_steps'],
        color=BLUE, linewidth=2.5, marker='o', markersize=7, label='Avg Steps')

ax2 = ax1.twinx()
ax2.plot(weekly['week_start'], weekly['active_users'],
        color=ORANGE, linewidth=2, marker='s', markersize=6,
        linestyle='--', label='Active Users')
ax2.set_ylabel('Active Users', color=ORANGE)
ax2.tick_params(axis='y', labelcolor=ORANGE)

ax1.set_title('Weekly Trend — User engagement drops over 4 weeks (case for gamification)')
ax1.set_xlabel('Week'); ax1.set_ylabel('Avg Steps', color=BLUE)
ax1.tick_params(axis='y', labelcolor=BLUE)
ax1.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:,.0f}'))
lines1, lbl1 = ax1.get_legend_handles_labels()
lines2, lbl2 = ax2.get_legend_handles_labels()
ax1.legend(lines1+lines2, lbl1+lbl2, loc='upper right')

```

```
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig('chart10_weekly_trend.png', bbox_inches='tight', dpi=150)
plt.show()
```



SECTION 6 — Sleep Analysis

CELL 14

Sleep Dashboard — 4 Panels — Chart 11 of 13

```
fig = plt.figure(figsize=(14, 10))
gs = gridspec.GridSpec(2, 2, hspace=0.4, wspace=0.35)

# Panel 1: Distribution of avg minutes asleep


---


ax1 = fig.add_subplot(gs[0,0])
ax1.hist(sleep['avg_min_asleep'], bins=14, color=PURPLE, edgecolor='white',
linewidth=0.9)
ax1.axvline(360, color=RED, linestyle='--', lw=1.4, label='6 hrs (360 min)')
ax1.axvline(420, color=GREEN, linestyle='--', lw=1.4, label='7 hrs target (420)')
ax1.axvline(480, color=ORANGE, linestyle='--', lw=1.4, label='8 hrs (480 min)')
ax1.set_title('Distribution - Avg Sleep per User')
ax1.set_xlabel('Minutes'); ax1.set_ylabel('Users'); ax1.legend(fontsize=8)

# Panel 2: Sleep efficiency vs duration scatter


---


ax2 = fig.add_subplot(gs[0,1])
sc = ax2.scatter(sleep['avg_min_asleep'], sleep['avg_sleep_efficiency'],
                 c=sleep['avg_min_wasted'], cmap='RdYlGn_r',
                 s=70, edgecolors='grey', linewidths=0.4)
fig.colorbar(sc, ax=ax2, label="Avg min awake in bed")
ax2.axhline(85, color=BLUE, linestyle='--', lw=1.2, label='85% efficiency
benchmark')
ax2.set_title('Sleep Efficiency vs Duration')
ax2.set_xlabel('Avg Minutes Asleep'); ax2.set_ylabel('Sleep Efficiency (%)')
ax2.legend(fontsize=8)

# Panel 3: % hitting 7hr goal by segment


---


ax3 = fig.add_subplot(gs[1,0])
merged = user_sum[['user_id', 'user_segment', 'avg_min_asleep']].dropna().copy()
merged['enough'] = merged['avg_min_asleep'] >= 420
seg_sl = (merged.groupby('user_segment')['enough'].mean()*100)
seg_sl = seg_sl.reindex(['Sedentary', 'Lightly Active', 'Fairly Active', 'Very
Active']).dropna()
b3 = ax3.bar(seg_sl.index, seg_sl.values,
             color=['#BFDBFE', '#60A5FA', '#2563EB', '#1F3864'], edgecolor='white')
for bar in b3:
    ax3.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
             f'{bar.get_height():.0f}%', ha='center', fontsize=10)
ax3.set_title('% Users Averaging ≥7hrs Sleep by Segment')
ax3.set_ylabel('% Users'); ax3.set_ylim(0,115);
ax3.tick_params(axis='x', rotation=20)

# Panel 4: Minutes awake in bed per user

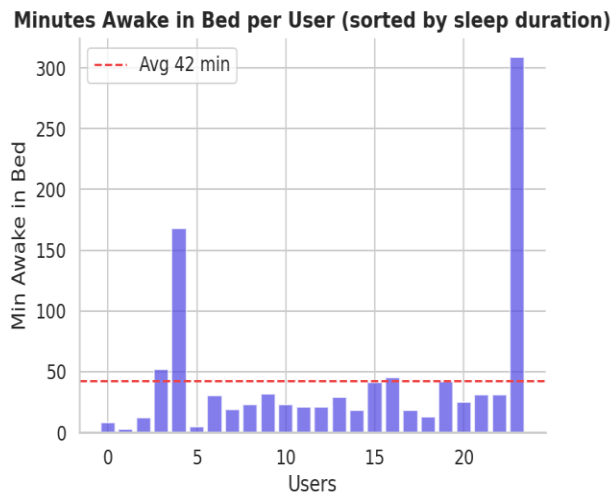
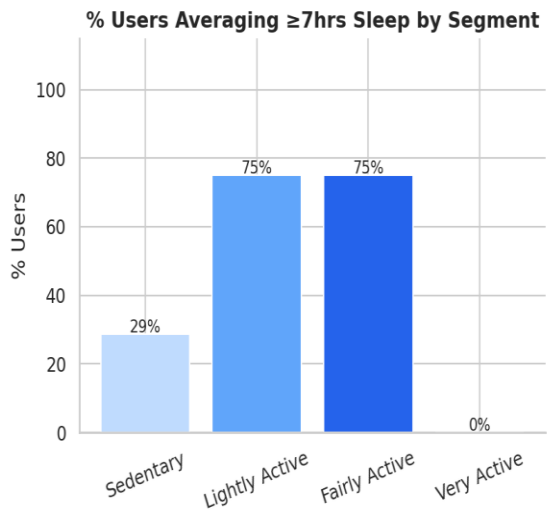
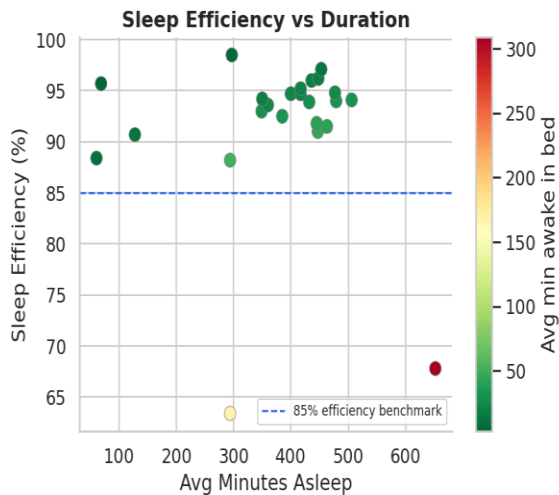
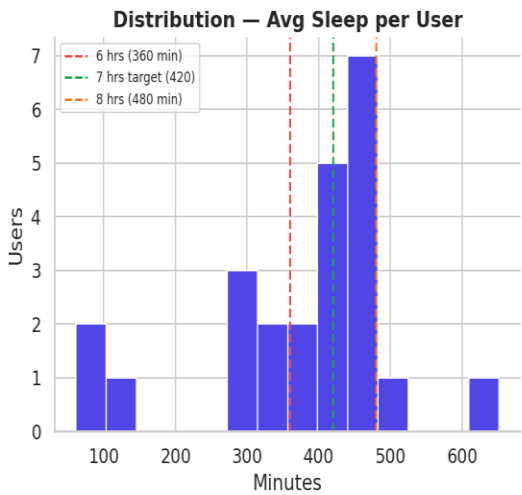

---


ax4 = fig.add_subplot(gs[1,1])
ss = sleep.sort_values('avg_min_asleep')
ax4.bar(range(len(ss)), ss['avg_min_wasted'], color=PURPLE, alpha=0.7,
edgecolor='white')
ax4.axhline(ss['avg_min_wasted'].mean(), color=RED, linestyle='--', lw=1.4,
            label=f"Avg {ss['avg_min_wasted'].mean():.0f} min")
ax4.set_title('Minutes Awake in Bed per User (sorted by sleep duration)')
ax4.set_xlabel('Users'); ax4.set_ylabel('Min Awake in Bed'); ax4.legend()

plt.suptitle('Sleep Quality Analysis Dashboard', fontsize=15, fontweight='bold',
y=1.01)
plt.savefig('chart11_sleep_dashboard.png', bbox_inches='tight', dpi=150)
plt.show()
```

```
print(f"% users ≥7hrs sleep : {(sleep['avg_min_asleep']>=420).mean()*100:.0f}%")
print(f"Avg sleep efficiency: {sleep['avg_sleep_efficiency'].mean():.1f}%")
```

Sleep Quality Analysis Dashboard



SECTION 7 — Sedentary Behaviour Analysis

CELL 15

Sedentary Time Deep-Dive — Chart 12 of 13

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Panel 1: Sedentary time vs calories


---


b1 = activity[['sedentary_minutes', 'calories']].dropna()
axes[0].scatter(b1['sedentary_minutes'], b1['calories'],
                alpha=0.25, s=14, color='#7C3AED')
r1 = pearsonr(b1['sedentary_minutes'], b1['calories'])[0]
axes[0].set_title(f'Sedentary Time vs Calories  r={r1:.3f}')
axes[0].set_xlabel('Sedentary Minutes'); axes[0].set_ylabel('Calories')

# Panel 2: Sedentary time vs steps


---


b2 = activity[['sedentary_minutes', 'total_steps']].dropna()
axes[1].scatter(b2['sedentary_minutes'], b2['total_steps'],
                alpha=0.25, s=14, color=RED)
r2 = pearsonr(b2['sedentary_minutes'], b2['total_steps'])[0]
axes[1].set_title(f'Sedentary Time vs Steps  r={r2:.3f}')
axes[1].set_xlabel('Sedentary Minutes'); axes[1].set_ylabel('Total Steps')
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:,.0f}'))

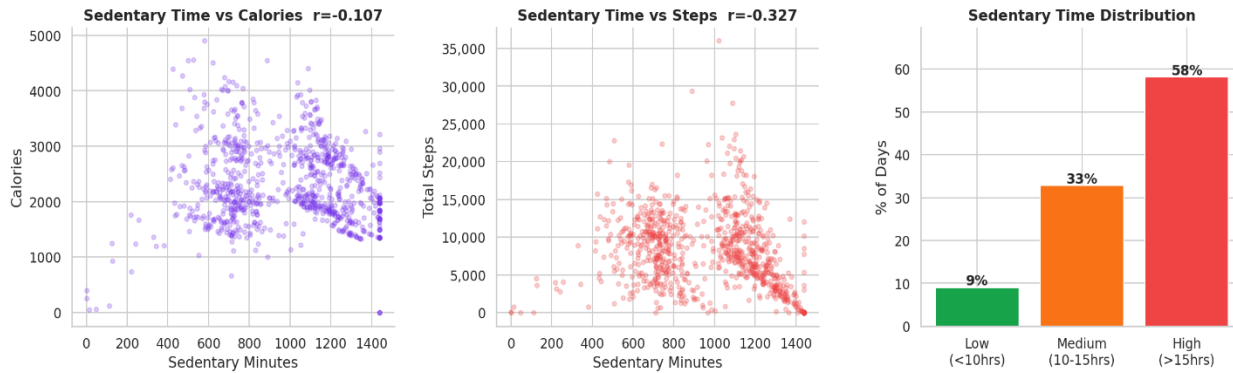
# Panel 3: Sedentary bucket distribution


---


activity['sed_bucket'] = pd.cut(
    activity['sedentary_minutes'],
    bins=[0, 600, 900, 1440],
    labels=['Low\n(<10hrs)', 'Medium\n(10-15hrs)', 'High\n(>15hrs)'])
bucket = activity['sed_bucket'].value_counts(normalize=True).sort_index() * 100
axes[2].bar(bucket.index, bucket.values,
            color=[GREEN, ORANGE, RED], edgecolor='white')
for i,v in enumerate(bucket.values):
    axes[2].text(i, v+0.5, f'{v:.0f}%', ha='center', fontsize=12,
                fontweight='bold')
axes[2].set_title('Sedentary Time Distribution')
axes[2].set_ylabel('% of Days'); axes[2].set_ylim(0, bucket.max()*1.2)

plt.suptitle('Sedentary Behaviour Deep-Dive', fontsize=14, fontweight='bold',
y=1.02)
plt.tight_layout()
plt.savefig('chart12_sedentary.png', bbox_inches='tight', dpi=150)
plt.show()
print(f"Days >15hrs sedentary:
      {(activity['sedentary_minutes']>=900).mean()*100:.0f}%")
print(f"Avg sedentary hrs/day: {activity['sedentary_minutes'].mean()/60:.1f}")
```

Sedentary Behaviour Deep-Dive



CELL 16

User Activity Heatmap — Chart 13 of 13

```
pivot = activity.pivot_table(
    index='user_id', columns='day_of_week',
    values='total_steps', aggfunc='mean')[DAY_ORDER].fillna(0)
pivot = pivot.loc[pivot.mean(axis=1).sort_values(ascending=False).index]

fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(
    pivot, ax=ax, cmap='Blues', linewidths=0.4, annot=False,
    cbar_kws={'label':'Avg Steps','shrink':0.6},
    xticklabels=[d[:3] for d in DAY_ORDER])
ax.set_title('User Activity Heatmap - Avg Steps by User & Day of Week',
             fontsize=13, fontweight='bold')
ax.set_xlabel('Day of Week'); ax.set_ylabel('User ID')
ax.set_yticklabels(ax.get_yticklabels(), fontsize=7)
plt.tight_layout()
plt.savefig('chart13_user_heatmap.png', bbox_inches='tight', dpi=150)
plt.show()
```

8877689391							
8053475328							
1503960366							
7007744171							
2022484408							
3977333714							
4388161847							
2347167796							
6962181067							
7086361926							
4702921684							
8378563200							
5553957443							
5577150313							
4558609924							
2873212765							
1644430081							
8583815059							
4319703577							
6117666160							
3372868164							
8253242879							
1624580081							
6290855005							
2026352035							
4445114986							
2320127002							
6775888955							
1844505072							
4057192912							
4020332650							
8792009665							
1927972279							

A vertical color scale bar on the right side of the figure. It transitions from light blue at the bottom to dark blue at the top. Numerical labels are placed to the right of the bar at intervals of 2,500, starting from 0 at the bottom and ending at 20,000 at the top.

Mon Tue Wed Thu Fri Sat Sun

Day of Week

SECTION 8 — Key Findings & Export

CELL 17

Key Findings Print Report

```
print('=' * 62)
print('    BELLABEAT FITNESS TRACKER — KEY FINDINGS')
print('=' * 62)

avg_steps = activity['total_steps'].mean()
avg_sed_hr = activity['sedentary_minutes'].mean()/60
pct_10k = (activity['total_steps']>=10000).mean()*100
pct_high_s = (activity['sedentary_minutes']>=900).mean()*100
peak_hour = hourly.loc[hourly['avg_steps'].idxmax(),'hour_of_day']
most_act = daytrend.loc[daytrend['avg_steps'].idxmax(),'day_of_week']
avg_eff = sleep['avg_sleep_efficiency'].mean()
pct_7hrs = (sleep['avg_min_asleep']>=420).mean()*100

print(f'\n❑ ACTIVITY')
print(f'    Avg daily steps      : {avg_steps:,.0f}    (WHO target: 10,000)')
print(f'    % days hitting 10k     : {pct_10k:.0f}%')
print(f'    Most active day        : {most_act}')
print(f'    Peak activity hour     : {peak_hour}:00')
print(f'\n❑ SEDENTARY BEHAVIOUR')
print(f'    Avg sedentary hrs/day  : {avg_sed_hr:.1f} hrs')
print(f'    Days >15hrs sedentary  : {pct_high_s:.0f}%')
print(f'\n❑ SLEEP (24 of 33 users logged sleep)')
print(f'    Avg sleep efficiency   : {avg_eff:.1f}% (benchmark: 85%)')
print(f'    % users avg ≥7hrs      : {pct_7hrs:.0f}%')
print(f'\n❑ BELLABEAT RECOMMENDATIONS')
print(f'    1. Push notifications at {peak_hour}:00 — peak activity window')
print(f'    2. Move-more alerts: {pct_high_s:.0f}% of days exceed 15hrs sedentary')
print(f'    3. Sleep coaching upsell: only {pct_7hrs:.0f}% hit the 7hr sleep goal')
print('=' * 62)
```

CELL 18

Export All 9 Power BI CSV Files + Charts

❑ **Insight:** This single cell produces all 9 pbi_ files needed for your Power BI dashboards. Run it last after all charts are done.

```
os.makedirs("powerbi_exports", exist_ok=True)

# — 1. pbi_master_activity.csv
_____
activity.to_csv('powerbi_exports/pbi_master_activity.csv', index=False)

# — 2. pbi_user_summary.csv
_____
user_sum.to_csv('powerbi_exports/pbi_user_summary.csv', index=False)

# — 3. pbi_user_segments.csv (built here — was the missing file)
_____
seg_export = (user_sum['user_segment']
              .value_counts()
              .reset_index()

              .rename(columns={"user_segment": "user_segment", "count": "user_count"}))
# Handle both old and new pandas column naming
if 'index' in seg_export.columns:
    seg_export =
seg_export.rename(columns={'index': 'user_segment', 'user_segment': 'user_count'})
```

```

seg_export.to_csv('powerbi_exports/pbi_user_segments.csv', index=False)

# — 4. pbi_day_trends.csv
daytrend.to_csv('powerbi_exports/pbi_day_trends.csv', index=False)

# — 5. pbi_peak_hours.csv
hourly.to_csv('powerbi_exports/pbi_peak_hours.csv', index=False)

# — 6. pbi_scatter.csv
scatter.to_csv('powerbi_exports/pbi_scatter.csv', index=False)

# — 7. pbi_sleep.csv
sleep.to_csv('powerbi_exports/pbi_sleep.csv', index=False)

# — 8. pbi_weekly_trend.csv
weekly.to_csv('powerbi_exports/pbi_weekly_trend.csv', index=False)

# — 9. pbi_hourly_combined.csv (same as peak_hours – for D3 notification bar)
hourly.to_csv('powerbi_exports/pbi_hourly_combined.csv', index=False)

# — Verify all 9 files
expected =
['pbi_master_activity.csv', 'pbi_user_summary.csv', 'pbi_user_segments.csv',
 'pbi_day_trends.csv', 'pbi_peak_hours.csv', 'pbi_scatter.csv',
 'pbi_sleep.csv', 'pbi_weekly_trend.csv', 'pbi_hourly_combined.csv']
print('✔ Power BI export files:')
for f in expected:
    path = f'powerbi_exports/{f}'
    kb = os.path.getsize(path)//1024 if os.path.exists(path) else 0
    ok = '✔' if os.path.exists(path) else '✗MISSING'
    print(f'    {ok}    {f:<40}    {kb:>5} KB')

```

CELL 19

Download Everything as ZIP in One Click

⚠ Note: Run this cell last. It zips all 13 charts + all 9 Power BI CSVs and downloads to your computer automatically.

```

# — Create zip file

with zipfile.ZipFile('bellabeat_all_outputs.zip', 'w', zipfile.ZIP_DEFLATED) as zf:
    # All 13 chart PNGs
    for f in sorted(os.listdir('.')):
        if f.startswith('chart') and f.endswith('.png'):
            zf.write(f)
            print(f'    Added chart: {f}')
    # All 9 Power BI CSV files
    for f in sorted(os.listdir('powerbi_exports')):
        zf.write(f'powerbi_exports/{f}', f)
        print(f'    Added PBI:    {f}')

print('\n✔ bellabeat_all_outputs.zip ready')

```

```
# — Auto-download
```

```
from google.colab import files
files.download('bellabeat_all_outputs.zip')
print('Downloading... check your browser downloads folder')
print('Extract the zip — upload the 9 pbi_ files to Power BI Desktop')
```

APPENDIX — Troubleshooting & Column Reference

Power BI file reference — what each pbi_ file feeds

File	Dashboard	Visual	Key columns used
pbi_user_segments.csv	D1	Donut chart	user_segment, user_count
pbi_scatter.csv	D1	Scatter chart	total_steps, calories, daily_activity_segment
pbi_day_trends.csv	D1	3 bar charts	day_of_week, avg_steps, avg_calories, avg_very_active_min
pbi_master_activity.csv	D1 D2 D3	KPI cards + slicer	all columns — primary source
pbi_user_summary.csv	D1 D2 D3	Slicer + gauge + activity score bar	user_segment, avg_daily_steps, avg_min_asleep, avg_activity_score
pbi_sleep.csv	D2	Sleep duration bar + efficiency scatter	user_id, avg_min_asleep, avg_sleep_efficiency, avg_min_wasted
pbi_peak_hours.csv	D3	Area chart	hour_of_day, avg_steps
pbi_hourly_combined.csv	D3	Notification timing bar	hour_of_day, avg_steps (grouped into windows)
pbi_weekly_trend.csv	D3	Weekly dual-axis line	week_start, avg_steps, active_users